PLD Compilation

H4214

30/03/2026

La Gestion de projet

Méthode Agile

Dans le cadre de ce projet, nous avons adopté une organisation basée sur une méthode agile, en travaillant par sprints:

- Le rendu intermédiaire
- IR du rendu intermédiaire
- Les fonctionnalités obligatoires
- Les fonctionnalités facultatives

Objectifs

🎯 *Obligatoire (Mandatory)*

x] Type de données de base `int` (un type 32 bits).
x] Déclaration de variables n'importe où.
x] Constantes entières .
x] Constantes caractère (avec simple quote).
x] Opérations arithmétiques de base (`+`, `-`, `*`).
x] Division et modulo.
x] Opérations logiques bit-à-bit (`|`, `&`, `^`).
x] Opérations de comparaison (`==`, `!=`, `<`, `>`).
X] Opérations unaires (`!`, `-`).
x] Affectation (qui retourne aussi une valeur).
x] Utilisation des fonctions standard `putchar` et `getchar` pour les entrées-sorties.
x] Définition de fonctions avec paramètres, et type de retour `int` ou `void`.
x] Vérification de la cohérence des appels de fonctions et leurs paramètres.
x] Structure de blocs grâce à `{` et `}`.
x] Support des portées de variables.
x] shadowing.
x] Les structures de contrôle `if`, `else`, `while`.
x] Support du `return expression` n'importe où.
x] Vérification qu'une variable utilisée a été déclarée.
x] Vérification qu'une variable n'est pas déclarée plusieurs fois.

🚀 *Facultatif (Optional)*

* [ ] Reciblage vers plusieurs architectures : x86, MSP430, ARM.
* [ ] Support des `double` avec toutes les conversions implicites.
* [ ] Propagation de constantes simple.
* [ ] Propagation de variables constantes (avec analyse du data-flow).
* [ ] Tableaux (à une dimension).
* [ ] Pointeurs.
* [x] `break` et `continue`.
* [ ] Les chaînes de caractères représentées par des tableaux de `char`.
* [x] Possibilité d'initialiser une variable lors de sa déclaration.
* [x] `switch...case`.
* [x] Les opérateurs logiques paresseux `||`, `&&`.
* [x] Opérateurs d'affectation (`+=`, `-=`) et d'incrémentation/décrémentation (`++`, `--`).

Le Visiteur de Variable

VariableVisitor :

- Effectue la première passe : analyse sémantique de l'AST
- Si des erreurs sont détectées => la compilation s'arrête
- Sinon, il transmet ses résultats au **CodeGenVisitor** (effectue 2ème passe) via **getFunctionStates()** et **getFunctionSignatures()**

```
ifccParser parser(&tokens);
tree::ParseTree *tree = parser.axiom();
if (parser.getNumberOfSyntaxErrors() != 0) {...}
...
VariableVisitor vv(debug);
vv.visit(tree);
if (vv.getErrorCount() > 0) {...}
...
CodeGenVisitor v(vv.getFunctionStates(), vv.getFunctionSignatures());
v.visit(tree);
```

Classes et structures :

- **VarInfo** : stocke l'offset pile, et si la variable a été lue / assignée
- **FunctionSemanticState** : table des variables + compteurs mémoire (une par fonction)
- **FunctionSignature** : type de retour et types des paramètres, pour valider les appels

```
struct FunctionSignature
{
    Type returnType;
    std::vector<Type> paramTypes;
};
```

```
struct varInfo
{
    int index;
    bool used;
    bool affected;
};
```

```
struct FunctionSemanticState
{
    std::map<std::string, varInfo> varTable;
    int nextIndex = 0;
    int compteurVar = 0;
    int declarationCounter = 0;
};
```

Table des variables :

- Types de donneés: void, int, char (<=> int)
- Chaque variable prend 4 octets

compteurVal >= nextIndex
La différence correspond aux octets réservés pour les temporaires

Chaque variable déclarée reçoit un nom unique (x#1, x#2...) pour gérer le shadowing

Offset de la prochaine case libre sur la pile

Taille totale à réserver sur la pile

```
struct FunctionSemanticState
{
    std::map<std::string, varInfo> varTable;
    int nextIndex = 0;
    int compteurVar = 0;
    int declarationCounter = 0;
};
```

Compteur qui s'incrémente à chaque déclaration de variable

Offset sur la pile

```
struct varInfo
{
    int index;
    bool used;
    bool affected;
};
```

Expressions temporaires :

```
void VariableVisitor::allocateTemporary()
{
    if (currentFunction.empty()){ return; }
    currentState().compteurVar += 4;
}
```

Vérification des erreurs :

Variables :

- Variable déclarée plusieurs fois dans le même bloc
- Variable utilisée avant déclaration ou avant affectation
- Variable déclarée mais jamais utilisée

Fonctions :

- Pas de main défini, ou main ne retourne pas int
- Fonction non-void sans return
- Fonction void qui retourne une valeur

Appels de fonctions :

- Fonction appelée mais non définie
- Mauvais nombre d'arguments
- Type d'argument incompatible

Flot de contrôle & types :

- break / continue hors d'une boucle
- Opération arithmétique ou comparaison sur une expression void

La Grammaire

Notre grammaire initiale

- Au bout de 2 séances
- Très axée sur le cours de Grammaire et Langages
- Peu lisible
- Complexe

```
axiom : prog EOF ;

prog : 'int' 'main' '(' ')' '{' declaration bloc return_stmt '}' ;

bloc : affectation bloc | /*epsilon*/ ;

declaration : 'int' declaration_var ';' | /*epsilon*/ ;
declaration_var : VAR ',' declaration_var | VAR ;
affectation : VAR '=' expression ';' ;

expression : '(' expression ')'                          #par
           | MINUS expression                            #opposite
           | expression op=(MUL|DIV|MOD) expression      #muldiv
           | expression op=(PLUS|MINUS) expression       #addsub
           | CONST                                       #const
           | VAR                                         #var;

return_stmt : RETURN expression ';' ;

RETURN : 'return' ;
CONST : [0-9]+ ;
COMMENT : '/*' .*? '*/' -> skip ;
DIRECTIVE : '#' .*? '\n' -> skip ;
WS    : [ \t\r\n] -> channel(HIDDEN);
VAR : [a-zA-Z][a-zA-Z0-9_]* ;
PLUS : '+';
MINUS : '-';
MUL : '*';
DIV : '/';
MOD : '%';
```

Notre grammaire actuelle

- Opérateurs Regex : *, + et ?
- Plus claire
- Fonctions et typage
- While, if, switch
- Gestion des priorités

```
axiom : prog EOF ;

prog : function_def+;

function_def : type VAR '(' param_list? ')' block;

param_list : param (',' param)*;

param : type VAR;

type : 'int' | 'void';

block
    : '{' stmt* '}'
    ;

stmt
    : declaration
    | CONTINUE ';'
    | BREAK ';'
    | return_stmt
    | block
    | expression ';'
    | conditional
    | while_conditional
    | switch_stmt
    ;
```

Nos opérateurs et tokens

```
expression  : '(' expression ')'                                  #par
            | NOT expression                                      #not
            | VAR PLUS PLUS                                       #post_incr
            | VAR MINUS MINUS                                     #post_decr
            | PLUS PLUS VAR                                       #pre_incr
            | MINUS MINUS VAR                                     #pre_decr
            | MINUS expression                                    #opposite
            | expression op=(MUL|DIV|MOD) expression              #muldiv
            | expression op=(PLUS|MINUS) expression               #addsub
            | expression op=(LTE|LT|GTE|GT) expression            #comp
            | expression op=(EQ|NEQ) expression                   #eq
            | expression BITWISE_AND expression                   #bitwise_and
            | expression BITWISE_XOR expression                   #bitwise_xor
            | expression BITWISE_OR expression                    #bitwise_or
            | expression LOGICAL_AND expression                   #logical_and
            | expression LOGICAL_OR expression                    #logical_or
            | VAR op=(AEQ|PEQ|MEQ) expression                     #affectation
            | VAR '(' arg_list? ')'                               #call
            | PUTCHAR '(' expression ')'                          #putchar
            | GETCHAR '(' ')'                                     #getchar
            | CONST                                               #const
            | CHARCONST                                           #charconst
            | VAR                                                 #var
            ;
```

```
COMMENT : '/*' .*? '*/' -> skip ;
DIRECTIVE : '#' .*? '\n' -> skip ;
WS    : [ \t\r\n] -> channel(HIDDEN);
PUTCHAR : 'putchar';
GETCHAR : 'getchar';
PLUS : '+';
MINUS : '-';
MUL : '*';
DIV : '/';
MOD : '%';
EQ : '==';
NEQ : '!=';
GT : '>';
GTE : '>=';
LT : '<';
LTE : '<=';
NOT : '!';
BITWISE_OR : '|';
BITWISE_AND : '&';
BITWISE_XOR : '^';
LOGICAL_AND : '&&';
LOGICAL_OR : '||';
AEQ : '=';
PEQ : '+=';
MEQ : '-=';
CONTINUE : 'continue';
BREAK : 'break';
CONST : [0-9]+ ;
VAR : [a-zA-Z][a-zA-Z0-9_]* ;
```

Le Générateur de Code

CodeGenVisitor

- Crée les variables temporaires
- Appelle l'IR avec le CFG
- Génère les prologues et épilogues du main

```
cfg.gen_asm_prologue(cout, currentFrameBytes, currentFunction == "main");

BasicBlock *entry = new BasicBlock(&cfg, cfg.new_BB_name());
cfg.add_bb(entry);
cfg.current_bb = entry;
scopeStack.push_back({});

if (ctx->param_list() != nullptr)
{
    for (ifccParser::ParamContext *param : ctx->param_list()->param())
    {
        visit(param);
    }
}
for (ifccParser::StmtContext *stmt : ctx->block()->stmt())
{
    visit(stmt);
}
if (!hasReturned && ctx->type()->getText() == "int")
{
    string zero = cfg.create_new_tempvar(Type::INT);
    cfg.add_to_symbol_table(zero, Type::INT);
    cfg.current_bb->add_IRInstr(IRInstr::ldconst, Type::INT, {zero, "0"});
    cfg.current_bb->add_IRInstr(IRInstr::return_instr, Type::INT, {zero});
    hasReturned = true;
}
scopeStack.pop_back();

cfg.gen_asm(cout);
cfg.gen_asm_epilogue(cout, currentFunction == "main");
```

Les opérateurs classiques

- Visite des expressions intermédiaires
- Crée des variables temporaires
- Génère les instructions
- Renvoie la variable temporaire finale

```
antlrcpp::Any CodeGenVisitor::visitAddsub(ifccParser::AddsubContext *ctx)
{
    char op = ctx->op->getText()[0];
    string lhs = std::any_cast<string>(visit(ctx->expression(0)));
    string rhs = std::any_cast<string>(visit(ctx->expression(1)));
    string out = cfg.create_new_tempvar(Type::INT);
    cfg.add_to_symbol_table(out, Type::INT);
    if (op == '+')
    {
        cfg.current_bb->add_IRInstr(IRInstr::add, Type::INT, {out, lhs, rhs});
    }
    else if (op == '-')
    {
        cfg.current_bb->add_IRInstr(IRInstr::sub, Type::INT, {out, lhs, rhs});
    }
    return out;
}
```

Autres opérateurs

- Les opérateurs de post-incrémentation
- Gestion des fonctions
- Gestion des blocs et des portées

```
antlrcpp::Any CodeGenVisitor::visitPost_incr(ifccParser::Post_incrContext *ctx)
{
    std::string varSource = ctx->VAR()->getText();
    std::string varName = resolveVisibleVar(varSource);
    postfixOps.push_back({varName, "++"});

    return varName;
}
```

```
antlrcpp::Any CodeGenVisitor::visitStmt(ifccParser::StmtContext *ctx)
{
    visitChildren(ctx);
    for (auto &op : postfixOps)
    {
        string tempVar = cfg.create_new_tempvar(Type::INT);
        cfg.add_to_symbol_table(tempVar, Type::INT);

        cfg.current_bb->add_IRInstr(IRInstr::ldconst, Type::INT, {tempVar, "1"});

        if (op.second == "++")
        {
            cfg.current_bb->add_IRInstr(IRInstr::add, Type::INT, {op.first, op.first, tempVar});
        }
        else if (op.second == "--")
        {
            cfg.current_bb->add_IRInstr(IRInstr::sub, Type::INT, {op.first, op.first, tempVar});
        }
    }
    postfixOps.clear();
    return antlrcpp::Any();
}
```

L’IR et les Opérateurs

Les classes

- Un CFG par fonction
- Un BasicBlock est une séquence linéaire avec exit_true/exit_false encodent tous les branchements

Operateurs IR

- Chaque opération IR correspond à une ou quelques instructions x86

- Pour les functions, les 6 premiers arguments dans%edi/%esi/%edx/%ecx/%r8d/%r9d, retour dans %eax (args>6 sur la pile expliquer plus tard)

Arithmetic		
+	add	addition
-	sub	subtraction
*	mul	multiply
/	div	divide
%	mod	modulo (→ %edx)
- (unary)	neg	negate

Bitwise & Logical		
&	bitwise_and	bitwise AND
|	bitwise_or	bitwise OR
^	bitwise_xor	bitwise XOR
!	bool_not	logical NOT → 0 or 1

Comparison (→ 0 or 1)		
==	cmp_eq	equal → sete
!=	cmp_neq	not equal → setne
<	cmp_lt	less than → setl
>	cmp_gt	greater than → setg
<=	cmp_elt	less or equal → setle
>=	cmp_egt	greater or equal → setge

Memory & Copy		
= const	ldconst	load constant
= var	copy	copy variable

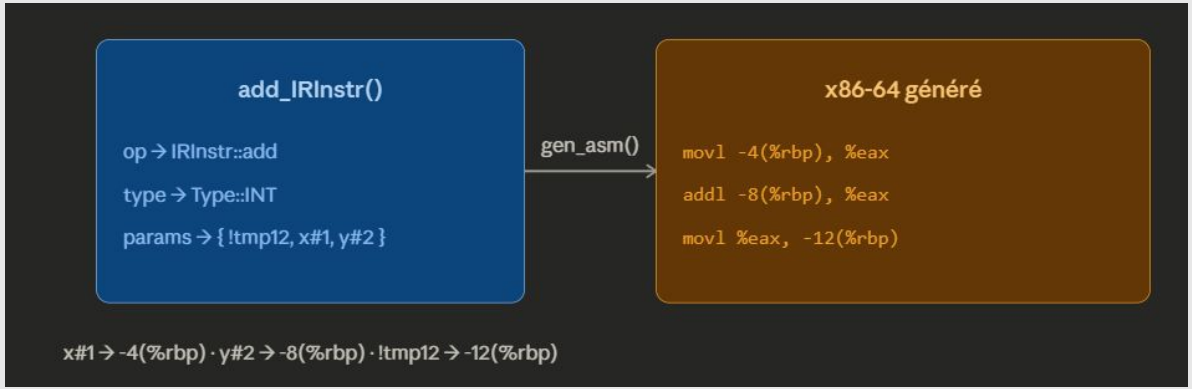
Functions		
param	load_param	receive argument
f(...)	call	function call
return	return_instr	return value

Control flow		
goto	jump	unconditional jump
getchar()	getchar	read char stdin
putchar()	putchar	write char stdout

Appel d'une IRInstr

- add_IRInstr encapsule le constructeur et ajoute l'instruction au bloc actif en un seul appel

- gen_asm() résout chaque nom en -N(%rbp) via la table des symboles, puis émet l'assembleur




```
string CFG::IR_reg_to_asm(string reg)
{
    return "-" + to_string(SymbolIndex[reg]) + "(%rbp)";
}
```

Encadrement de functions

- Le prologue sauvegarde %rbp, crée le frame pointer et réserve l'espace pile pour toutes les variables locales

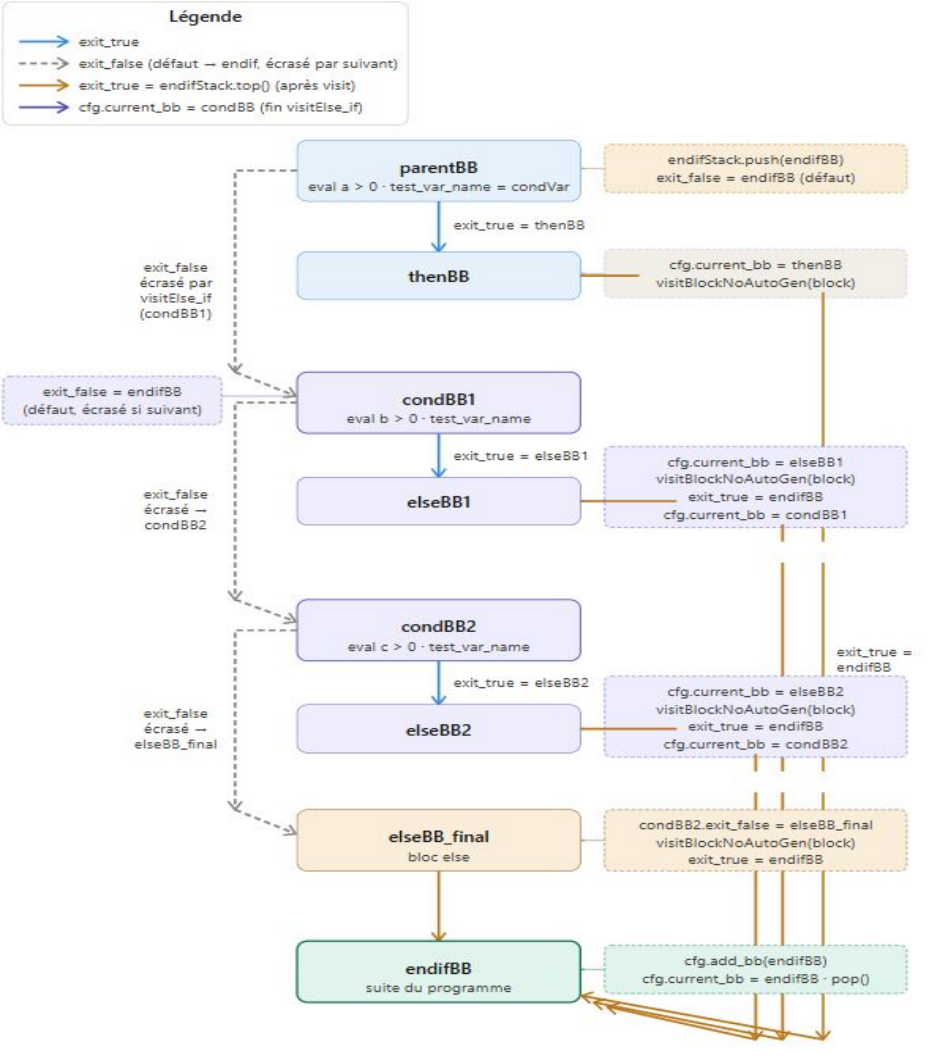
- Toutes les instructions return_instr sautent vers returnLabel

- L'épilogue restaure exactement l'état d'avant l'appel : %rsp, %rbp, puis ret

```
void CFG::gen_asm_prologue(ostream &o, int compteurVar, bool emitGlobal)
{
#ifdef __APPLE__
    if (emitGlobal)
    {
        o << ".globl _" << functionName << "\n";
    }
    o << " _" << functionName << ": \n";
#else
    if (emitGlobal)
    {
        o << ".globl " << functionName << "\n";
    }
    o << " " << functionName << ": \n";
#endif

    o << "    pushq %rbp\n";
    o << "    movq %rsp, %rbp\n";
    o << "    subq $" << compteurVar << ", %rsp" << endl;
}
void CFG::gen_asm_epilogue(ostream &o, bool emitStackNote)
{
    o << returnLabel << ":" << endl;
    o << "    movq %rbp, %rsp\n";
    o << "    popq %rbp\n";
    o << "    ret\n";

#ifndef __APPLE__
    if (emitStackNote)
    {
        o << ".section .note.GNU-stack,\"\",@progbits\n";
    }
#endif
}
```

La gestion des blocs et boucles

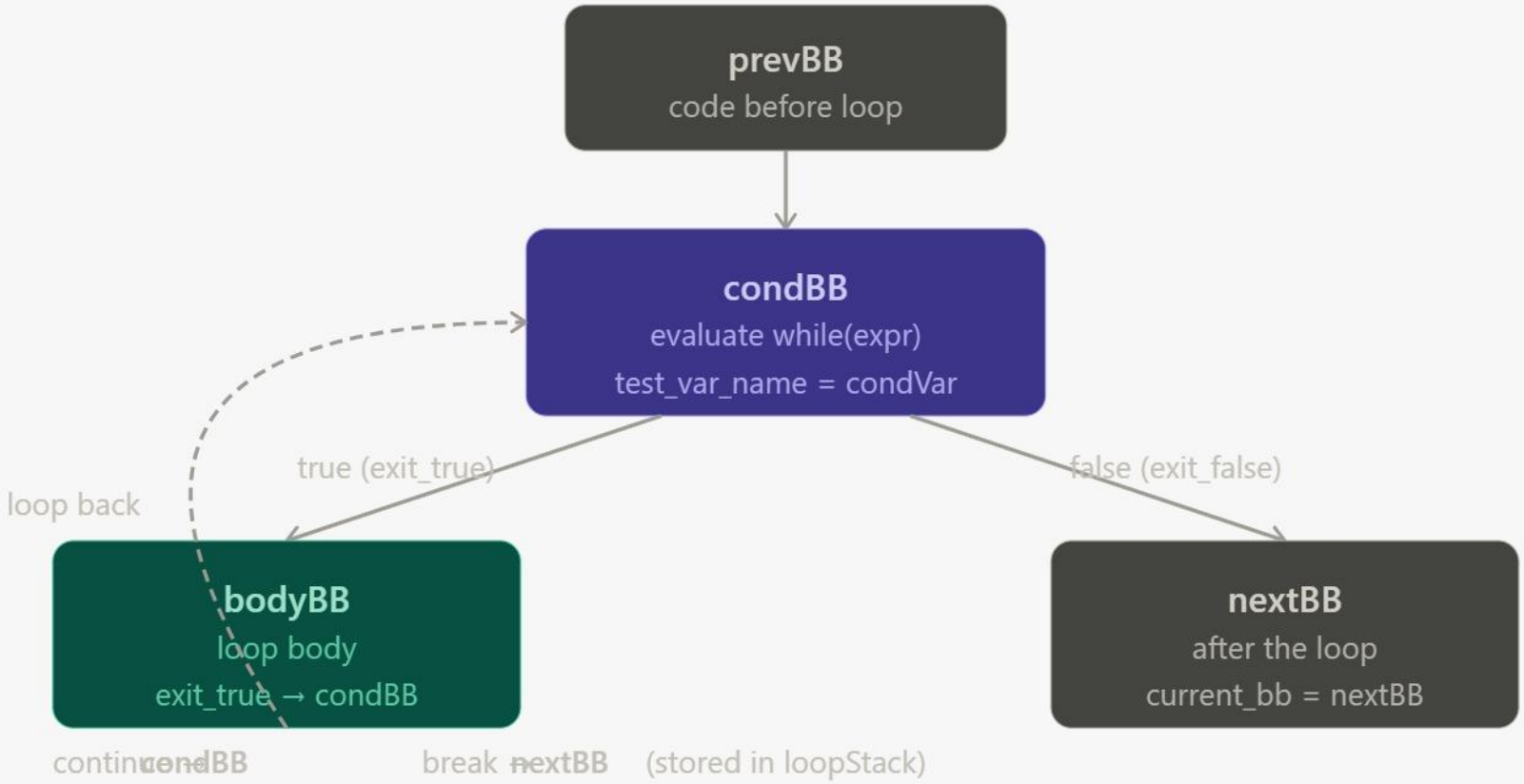
```
antlrcpp::Any CodeGenVisitor::visitConditional(ifccParser::ConditionalContext *ctx)
{

    BasicBlock *endifBB = new BasicBlock(&cfg, cfg.new_BB_name());
    endifStack.push(endifBB);

    string condVar = std::any_cast<string>(visit(ctx->expression()));
    cfg.current_bb->test_var_name = condVar; // le parentBB a la condition

    BasicBlock *thenBB = new BasicBlock(&cfg, cfg.new_BB_name());
    cfg.add_bb(thenBB);

    cfg.current_bb->exit_true = thenBB;            // l'exit true du block parent est toujours thenBB
    cfg.current_bb->exit_false = endifStack.top(); // pointe par default vers le endif, peut être changé par un autre passage dans else/else_if

    for (auto s : ctx->else_stmt()) // visiter tout les else, else if
    {
        visit(s);
    }

    cfg.current_bb = thenBB;
    visitBlockNoAutoGen(ctx->block()); // visiter tout ce qu'il y as dans le bloc du then

    cfg.current_bb->exit_true = endifStack.top(); // jump inconditionelle de n'importe quoi qui vient aprés le if

    cfg.add_bb(endifStack.top());
    cfg.current_bb = endifStack.top(); // the rest of the program will add to the endif block

    endifStack.pop();

    return 0;
}
```


Légende
exit_true
exit_false (défaut → endif, écrasé par suivant)
exit_true = endifStack.top() (après visit)
cfg.current_bb = condBB (fin visitElse_if)
parentBB
eval a > 0 · test_var_name = condVar
endifStack.push(endifBB)
exit_false = endifBB (défaut)
exit_true = thenBB
thenBB
cfg.current_bb = thenBB
visitBlockNoAutoGen(block)
exit_false
écrasé par
visitElse_if
(condBB1)
exit_false = endifBB
(défaut, écrasé si suivant)
condBB1
eval b > 0 · test_var_name
exit_true = elseBB1
elseBB1
cfg.current_bb = elseBB1
visitBlockNoAutoGen(block)
exit_true = endifBB
cfg.current_bb = condBB1
exit_false
écrasé →
condBB2
condBB2
eval c > 0 · test_var_name
exit_true =
endifBB
exit_true = elseBB2
elseBB2
cfg.current_bb = elseBB2
visitBlockNoAutoGen(block)
exit_true = endifBB
cfg.current_bb = condBB2
exit_false
écrasé →
elseBB_final
elseBB_final
bloc else
condBB2.exit_false = elseBB_final
visitBlockNoAutoGen(block)
exit_true = endifBB
endifBB
suite du programme
cfg.add_bb(endifBB)
cfg.current_bb = endifBB · pop()

```
antlrcpp::Any CodeGenVisitor::visitElse_if(ifccParser::Else_ifContext *ctx)
{
    string condVar = std::any_cast<string>(visit(ctx->expression()));
    BasicBlock *elseBB = new BasicBlock(&cfg, cfg.new_BB_name());
    BasicBlock *condBB = new BasicBlock(&cfg, cfg.new_BB_name());

    condBB->test_var_name = condVar;
    condBB->exit_true = elseBB;
    condBB->exit_false = endifStack.top(); // condBB pointe par default vers le endif, peut être changé par un autre passage dans cette fonction ou la fonction else

    cfg.current_bb->exit_false = condBB; // on sait que le current_bb c'est soit le parentBB soit le condBB précedent(là ou la ligne juste en haut est changée)

    cfg.add_bb(elseBB);
    cfg.add_bb(condBB);

    cfg.current_bb = elseBB;
    visitBlockNoAutoGen(ctx->block());            // on explore tout ce qu'il y as dans le block de code du else if
    cfg.current_bb->exit_true = endifStack.top(); // le dernier block pointe vers le top du endifStack

    cfg.current_bb = condBB; // on met comme current block le condBB pour changer son exit false au passage suivant

    return 0;
}

antlrcpp::Any CodeGenVisitor::visitElse(ifccParser::ifccParser::ElseContext *ctx)
{

    BasicBlock *elseBB = new BasicBlock(&cfg, cfg.new_BB_name());

    cfg.add_bb(elseBB);
    cfg.current_bb->exit_false = elseBB; // parent or last cond block

    cfg.current_bb = elseBB;
    visitBlockNoAutoGen(ctx->block());
    cfg.current_bb->exit_true = endifStack.top();

    return 0;
}
```

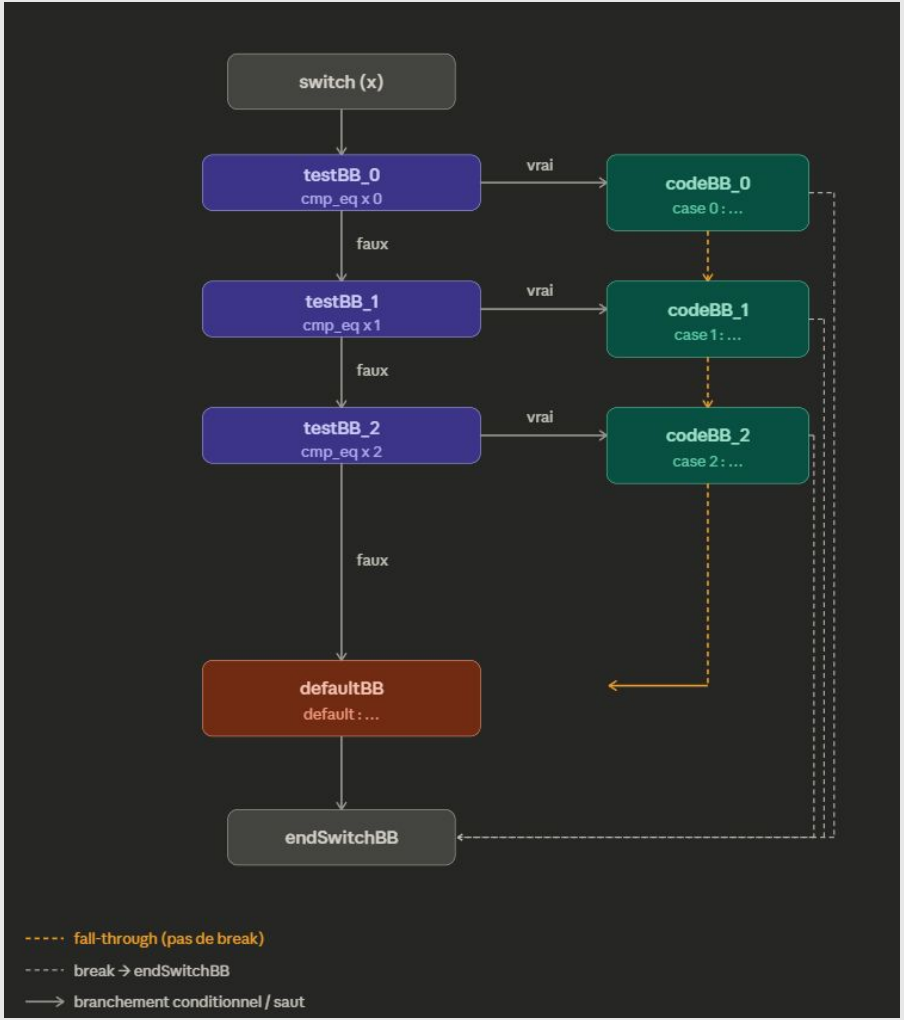
```cpp
antlrcpp::Any CodeGenVisitor::visitWhile_conditional(ifccParser::While_conditionalContext *ctx)
{
    BasicBlock *condBB = new BasicBlock(&cfg, cfg.new_BB_name());
    cfg.add_bb(condBB);
    BasicBlock *bodyBB = new BasicBlock(&cfg, cfg.new_BB_name());
    cfg.add_bb(bodyBB);
    BasicBlock *nextBlock = new BasicBlock(&cfg, cfg.new_BB_name());
    cfg.add_bb(nextBlock);

    pair<BasicBlock *, BasicBlock *> newLoop;
    newLoop.first = condBB;
    newLoop.second = nextBlock;
    loopStack.push_back(newLoop);

    cfg.current_bb->exit_true = condBB; // jump inconditionel du bloc parent vers le condBB

    cfg.current_bb = condBB;
    string condVar = std::any_cast<string>(visit(ctx->expression()));

    cfg.current_bb->test_var_name = condVar;
    cfg.current_bb->exit_true = bodyBB;
    cfg.current_bb->exit_false = nextBlock;

    cfg.current_bb = bodyBB;
    visitBlockNoAutoGen(ctx->block()); // visiter le bloc dans code dans le body

    cfg.current_bb->exit_true = condBB; // le dernier block su code de body pointe vers le condBB
    cfg.current_bb = nextBlock;

    loopStack.pop_back();

    return 0;
}
```

```cpp
antlrcpp::Any CodeGenVisitor::visitStmt(ifccParser::StmtContext *ctx)
{
    if (ctx->CONTINUE() != nullptr)
    {
        cfg.current_bb->add_IRInstr(IRInstr::jump, Type::INT, {loopStack.back().first->label});
    }
    else if (ctx->BREAK() != nullptr)
    {
        cfg.current_bb->add_IRInstr(IRInstr::jump, Type::INT, {loopStack.back().second->label});
    }
```


prevBB
code before loop
condBB
evaluate while(expr)
test_var_name = condVar
true (exit_true)
false (exit_false)
loop back
bodyBB
loop body
exit_true → condBB
nextBB
after the loop
current_bb = nextBB
continue condBB
break nextBB
(stored in loopStack)

```
antlrcpp::Any CodeGenVisitor::visitSwitch_stmt(ifccParser::Switch_stmtContext *ctx)
{
    string switchVar = std::any_cast<string>(visit(ctx->expression()));
    BasicBlock *endSwitchBB = new BasicBlock(&cfg, cfg.new_BB_name());
    cfg.add_bb(endSwitchBB);

    // On peut avoir des break mais pas de continue
    loopStack.push_back({nullptr, endSwitchBB});

    auto cases = ctx->switch_case();
    int casesNumber = cases.size();

    // On crée les blocs de code
    vector<BasicBlock*> codeBBs;
    for (int i = 0; i < casesNumber; i++) {
        codeBBs.push_back(new BasicBlock(&cfg, cfg.new_BB_name()));
        cfg.add_bb(codeBBs[i]);
    }
    BasicBlock *defaultBB = nullptr;
    if (ctx->switch_default()) {
        defaultBB = new BasicBlock(&cfg, cfg.new_BB_name());
        cfg.add_bb(defaultBB);
    }
```

```
BasicBlock *currentTestBB = cfg.current_bb;
for (int i = 0; i < casesNumber; i++)+
{
    BasicBlock *nextTestBB = new BasicBlock(&cfg, cfg.new_BB_name());
    cfg.add_bb(nextTestBB);

    cfg.current_bb = currentTestBB;

    string caseVal = cases[i]->CONST()->getText();
    string caseValLoc = cfg.create_new_tempvar(Type::INT);
    cfg.add_to_symbol_table(caseValLoc, Type::INT);
    cfg.current_bb->add_IRInstr(IRInstr::ldconst, Type::INT, {caseValLoc, caseVal});

    string tmpVar = cfg.create_new_tempvar(Type::INT);
    cfg.add_to_symbol_table(tmpVar, Type::INT);

    currentTestBB->add_IRInstr(IRInstr::cmp_eq, Type::INT, {tmpVar, switchVar, caseValLoc});
    currentTestBB->test_var_name = tmpVar;
    currentTestBB->exit_true = codeBBs[i];
    currentTestBB->exit_false = nextTestBB;

    cfg.current_bb = codeBBs[i];
    for (auto stmt : cases[i]->stmt()) { visit(stmt); }

    if (cfg.current_bb->exit_true == nullptr) {
        if (i + 1 < casesNumber) {
            cfg.current_bb->exit_true = codeBBs[i+1];
        } else {
            cfg.current_bb->exit_true = (defaultBB) ? defaultBB : endSwitchBB;
        }
    }
    currentTestBB = nextTestBB;
}

if (defaultBB) {
    currentTestBB->exit_true = defaultBB;
    cfg.current_bb = defaultBB;
    for (auto stmt : ctx->switch_default()->stmt()) { visit(stmt); }
    if (cfg.current_bb->exit_true == nullptr) { cfg.current_bb->exit_true = endSwitchBB; }
} else {
    currentTestBB->exit_true = endSwitchBB;
}

cfg.current_bb = endSwitchBB;
loopStack.pop_back();
return 0;
}
```


switch (x)
testBB_0
cmp_eq x 0
vrai
codeBB_0
case 0 : ...
faux
testBB_1
cmp_eq x 1
vrai
codeBB_1
case 1 : ...
faux
testBB_2
cmp_eq x 2
vrai
codeBB_2
case 2 : ...
faux
defaultBB
default : ...
endSwitchBB
fall-through (pas de break)
break → endSwitchBB
branchement conditionnel / saut

Shadowing

Shadowing

{nom_brut : nom_scoped, scope_lvl}

{“x” : “x#2”, 2}

{“x” : “x#1”, 1}

{{(“x”, “x#1”)}, {(“x”, “x#2”)}}

```
1 int main() {
2     int x;
3     x = 1;
4     {
5         int x;
6         x = 2;
7     }
8     return x;
9 }
```

Shadowing

```
std::string VariableVisitor::declareVar(const std::string &varName, bool affected)
{
    if (checkIfVarInCurrentBlock(varName))
    {
        reportError("La variable " + varName + " est declaree plusieurs fois.");
        return "";
    }

    FunctionSemanticState &state = currentState();
    state.declarationCounter++;
    std::string symbolName = varName + "#" + std::to_string(state.declarationCounter);

    varInfo info;
    info.index = state.nextIndex;
    state.nextIndex += 4;
    state.compteurVar += 4;
    info.used = false;
    info.affected = affected;
    state.varTable[symbolName] = info;

    scopedVars.push({varName, symbolName, scopeBlock});

    if (debug)
    {
        std::cout << "declaration de " << varName << " (" << symbolName << ") : " << info.index << std::endl;
    }

    return symbolName;
}
```

```
std::string VariableVisitor::resolveVisibleVarSymbol(const std::string &var) const
{
    std::stack<ScopedVarInfo> temp = scopedVars;
    while (!temp.empty())
    {
        auto top = temp.top();
        if (top.name == var)
        {
            return top.symbol;
        }
        temp.pop();
    }
    return "";
}
```

```
std::string CodeGenVisitor::resolveVisibleVar(const std::string &name) const
{
    for (auto it = scopeStack.rbegin(); it != scopeStack.rend(); ++it)
    {
        auto found = it->find(name);
        if (found != it->end())
        {
            return found->second;
        }
    }
    return "";
}
```

Les fonctions

Calls

```
std::any VariableVisitor::visitCall(ifccParser::CallContext *ctx)
{
    std::string functionName = ctx->VAR()->getText();
    auto it = functionSignatures.find(functionName);
    if (it == functionSignatures.end())
    {
        reportError("La fonction " + functionName + " n'est pas definie.");
        return Type::INT;
    }

    std::vector<Type> args;
    if (ctx->arg_list() != nullptr)
    {
        for (ifccParser::ExpressionContext *argExpr : ctx->arg_list()->expression())
        {
            args.push_back(evalExpr(argExpr));
        }
    }

    const FunctionSignature &sig = it->second;
    if (args.size() != sig.paramTypes.size())
    {
        reportError("La fonction " + functionName + " attend " + std::to_string(sig.paramTypes.size()) + " argument(s), mais " + std::to_string(args.size()) + " ont ete fournis.");
    }

    size_t common = std::min(args.size(), sig.paramTypes.size());
    for (size_t i = 0; i < common; ++i)
    {
        if (args[i] == Type::VOID || sig.paramTypes[i] != args[i])
        {
            reportError("Type incompatible pour l'argument " + std::to_string(i + 1) + " de " + functionName + ".");
        }
    }

    if (sig.returnType != Type::VOID)
    {
        allocateTemporary();
    }
    return sig.returnType;
}
```

Calls

```
antlrcpp::Any CodeGenVisitor::visitCall(ifccParser::CallContext *ctx)
{
    string out = cfg.create_new_tempvar(Type::INT);
    cfg.add_to_symbol_table(out, Type::INT);

    vector<string> params;
    params.push_back(ctx->VAR()->getText());
    params.push_back(out);
    if (ctx->arg_list() != nullptr)
    {
        for (ifccParser::ExpressionContext *arg : ctx->arg_list()->expression())
        {
            params.push_back(std::any_cast<string>(visit(arg)));
        }
    }
    cfg.current_bb->add_IRInstr(IRInstr::call, Type::INT, params);

    auto sigIt = functionSignatures.find(ctx->VAR()->getText());
    if (sigIt != functionSignatures.end() && sigIt->second.returnType == Type::VOID)
    {
        return string("");
    }

    return out;
}
```

```
antlrcpp::Any CodeGenVisitor::visitFunction_def(ifccParser::Function_defContext *ctx)
{
    bindFunctionState(ctx->VAR()->getText());
    hasReturned = false;

    cfg = CFG();
    cfg.set_function_name(currentFunction);
    cfg.current_bb = nullptr;

    for (const auto &[name, info] : currentVarTable)
    {
        cfg.add_to_symbol_table(name, Type::INT);
    }

    cfg.gen_asm_prologue(cout, currentFrameBytes, currentFunction == "main");

    BasicBlock *entry = new BasicBlock(&cfg, cfg.new_BB_name());
    cfg.add_bb(entry);
    cfg.current_bb = entry;
    scopeStack.push_back({});

    if (ctx->param_list() != nullptr)
    {
        for (ifccParser::ParamContext *param : ctx->param_list()->param())
        {
            visit(param);
        }
    }
    for (ifccParser::StmtContext *stmt : ctx->block()->stmt())
    {
        visit(stmt);
    }

    if (!hasReturned && ctx->type()->getText() == "int")
    {
        string zero = cfg.create_new_tempvar(Type::INT);
        cfg.add_to_symbol_table(zero, Type::INT);
        cfg.current_bb->add_IRInstr(IRInstr::ldconst, Type::INT, {zero, "0"});
        cfg.current_bb->add_IRInstr(IRInstr::return_instr, Type::INT, {zero});
        hasReturned = true;
    }

    scopeStack.pop_back();

    cfg.gen_asm(cout);
    cfg.gen_asm_epilogue(cout, currentFunction == "main");

    return antlrcpp::Any();
}
```

```
antlrcpp::Any CodeGenVisitor::visitParam(ifccParser::ParamContext *ctx)
{
    std::string sourceName = ctx->VAR()->getText();
    std::string scopedName = createScopedName(sourceName);
    scopeStack.back()[sourceName] = scopedName;

    cfg.current_bb->add_IRInstr(IRInstr::load_param, Type::INT, {scopedName, std::to_string(currentParamIndex)});
    currentParamIndex++;

    return antlrcpp::Any();
}
```

```
case call:
{
    for (int i = 0; i < params.size() - 2 && i < 6; i++)
    {
        o << "    movl " << bb->cfg->IR_reg_to_asm(params[i + 2]) << ", " << regs32[i] << endl;
    }

    int extraArgs = static_cast<int>(params.size()) - 8;
    if (extraArgs > 0)
    {
        for (int i = static_cast<int>(params.size()) - 1; i >= 8; --i)
        {
            o << "    movl " << bb->cfg->IR_reg_to_asm(params[i]) << ", %eax" << endl;
            o << "    pushq %rax" << endl;
        }
    }

    o << "    call " << params[0] << endl;

    if (extraArgs > 0)
    {
        o << "    addq $" << (extraArgs * 8) << ", %rsp" << endl;
    }

    o << "    movl %eax, " << bb->cfg->IR_reg_to_asm(params[1]) << endl;
    break;
}
case load_param:
{
    int paramIndex = stoi(params[1]);
    if (paramIndex < 6)
    {
        o << "    movl " << regs32[paramIndex] << ", " << bb->cfg->IR_reg_to_asm(params[0]) << endl;
    }
    else
    {
        int stackOffset = 16 + ((paramIndex - 6) * 8);
        o << "    movl " << stackOffset << "(%rbp), %eax" << endl;
        o << "    movl %eax, " << bb->cfg->IR_reg_to_asm(params[0]) << endl;
    }
    break;
}
```

```
add:
    pushq %rbp
    movq %rsp, %rbp
    subq $16, %rsp
add_bb0:
    movl %edi, -4(%rbp)
    movl %esi, -8(%rbp)
    movl -4(%rbp), %eax
    addl -8(%rbp), %eax
    movl %eax, -12(%rbp)
    movl -12(%rbp), %eax
    jmp add_return_exit
add_return_exit:
    movq %rbp, %rsp
    popq %rbp
    ret
subtract:
    pushq %rbp
    movq %rsp, %rbp
    subq $16, %rsp
subtract_bb0:
    movl %edi, -4(%rbp)
    movl %esi, -8(%rbp)
    movl -4(%rbp), %eax
    subl -8(%rbp), %eax
    movl %eax, -12(%rbp)
    movl -12(%rbp), %eax
    jmp subtract_return_exit
subtract_return_exit:
    movq %rbp, %rsp
    popq %rbp
    ret
```

```
.globl main
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $28, %rsp
main_bb0:
    movl $5, -12(%rbp)
    movl $3, -16(%rbp)
    movl -12(%rbp), %edi
    movl -16(%rbp), %esi
    call add
    movl %eax, -8(%rbp)
    movl -8(%rbp), %eax
    movl %eax, -4(%rbp)
    movl $1, -24(%rbp)
    movl -4(%rbp), %edi
    movl -24(%rbp), %esi
    call subtract
    movl %eax, -20(%rbp)
    movl -20(%rbp), %eax
    movl %eax, -4(%rbp)
    movl -4(%rbp), %eax
    jmp main_return_exit
main_return_exit:
    movq %rbp, %rsp
    popq %rbp
    ret
```

```c
int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}

int main() {
    int x;
    x = add(5, 3);
    x = subtract(x, 1);
    return x;
}
```

```
add:
    pushq %rbp
    movq %rsp, %rbp
    subq $16, %rsp
add_bb0:
    movl %edi, -4(%rbp)
    movl %esi, -8(%rbp)
    movl -4(%rbp), %eax
    addl -8(%rbp), %eax
    movl %eax, -12(%rbp)
    movl -12(%rbp), %eax
    jmp add_return_exit
add_return_exit:
    movq %rbp, %rsp
    popq %rbp
    ret
subtract:
    pushq %rbp
    movq %rsp, %rbp
    subq $16, %rsp
subtract_bb0:
    movl %edi, -4(%rbp)
    movl %esi, -8(%rbp)
    movl -4(%rbp), %eax
    subl -8(%rbp), %eax
    movl %eax, -12(%rbp)
    movl -12(%rbp), %eax
    jmp subtract_return_exit
subtract_return_exit:
    movq %rbp, %rsp
    popq %rbp
    ret
```

```
.globl main
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $28, %rsp
main_bb0:
    movl $5, -12(%rbp)
    movl $3, -16(%rbp)
    movl -12(%rbp), %edi
    movl -16(%rbp), %esi
    call add
    movl %eax, -8(%rbp)
    movl -8(%rbp), %eax
    movl %eax, -4(%rbp)
    movl $1, -24(%rbp)
    movl -4(%rbp), %edi
    movl -24(%rbp), %esi
    call subtract
    movl %eax, -20(%rbp)
    movl -20(%rbp), %eax
    movl %eax, -4(%rbp)
    movl -4(%rbp), %eax
    jmp main_return_exit
main_return_exit:
    movq %rbp, %rsp
    popq %rbp
    ret
```

```c
int sum7(int a, int b, int c, int d, int e, int f, int g)
{
    return a + b + c + d + e + f + g;
}

int main()
{
    return sum7(1, 2, 3, 4, 14, 10, 8);
}
```

```
sum7:
    pushq %rbp
    movq %rsp, %rbp
    subq $56, %rsp
sum7_bb0:
    movl %edi, -4(%rbp)
    movl %esi, -8(%rbp)
    movl %edx, -12(%rbp)
    movl %ecx, -16(%rbp)
    movl %r8d, -20(%rbp)
    movl %r9d, -24(%rbp)
    movl 16(%rbp), %eax
    movl %eax, -28(%rbp)
    movl -4(%rbp), %eax
    addl -8(%rbp), %eax
    movl %eax, -32(%rbp)
    movl -32(%rbp), %eax
    addl -12(%rbp), %eax
    movl %eax, -36(%rbp)
    movl -36(%rbp), %eax
    addl -16(%rbp), %eax
    movl %eax, -40(%rbp)
    movl -40(%rbp), %eax
    addl -20(%rbp), %eax
    movl %eax, -44(%rbp)
    movl -44(%rbp), %eax
    addl -24(%rbp), %eax
    movl %eax, -48(%rbp)
    movl -48(%rbp), %eax
    addl -28(%rbp), %eax
    movl %eax, -52(%rbp)
    movl -52(%rbp), %eax
    jmp sum7_return_exit
sum7_return_exit:
    movq %rbp, %rsp
    popq %rbp
    ret
```

```
.globl main
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $36, %rsp
main_bb0:
    movl $1, -8(%rbp)
    movl $2, -12(%rbp)
    movl $3, -16(%rbp)
    movl $4, -20(%rbp)
    movl $14, -24(%rbp)
    movl $10, -28(%rbp)
    movl $8, -32(%rbp)
    movl -8(%rbp), %edi
    movl -12(%rbp), %esi
    movl -16(%rbp), %edx
    movl -20(%rbp), %ecx
    movl -24(%rbp), %r8d
    movl -28(%rbp), %r9d
    movl -32(%rbp), %eax
    pushq %rax
    call sum7
    addq $8, %rsp
    movl %eax, -4(%rbp)
    movl -4(%rbp), %eax
    jmp main_return_exit
main_return_exit:
    movq %rbp, %rsp
    popq %rbp
    ret
```

Doubles (abandonnés)

```c
int main() {
        double a = 5.2, b = 3.0, c = 6.3;
        return a;
}
```

```
.section .text
.globl main
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $56, %rsp
main_bb0:
    movsd .LC1(%rip), %xmm0
    movsd %xmm0, -28(%rbp)
    movsd -28(%rbp), %xmm0
    movsd %xmm0, -4(%rbp)
    movsd .LC2(%rip), %xmm0
    movsd %xmm0, -36(%rbp)
    movsd -36(%rbp), %xmm0
    movsd %xmm0, -12(%rbp)
    movsd .LC3(%rip), %xmm0
    movsd %xmm0, -44(%rbp)
    movsd -44(%rbp), %xmm0
    movsd %xmm0, -20(%rbp)
    movsd -4(%rbp), %xmm0
    cvttsd2si %xmm0, %eax
    movl %eax, -52(%rbp)
    movl -52(%rbp), %eax
    jmp main_return_exit
main_return_exit:
    movq %rbp, %rsp
    popq %rbp
    ret
.section .note.GNU-stack,"",@progbits
.section .rodata
.LC0:
    .double 0
.LC1:
    .double 5.2
.LC2:
    .double 3
.LC3:
    .double 6.3
```

```c
int main() {
    double a;
    a = 3.0 + 1;
    return a;
}
```

Renvoie 4,
Ça marche

```c
int main() {
    double a;
    a = 1 + 3.0;
    return a;
}
```

Renvoie 3,
Ça ne marche pas

```
.section .text
.globl main
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $44, %rsp
main_bb0:
    movl $1, -12(%rbp)
    movsd .LC1(%rip), %xmm0
    movsd %xmm0, -16(%rbp)
    movl -12(%rbp), %eax
    cvtsi2sd %eax, %xmm0
    movsd %xmm0, -24(%rbp)
    movsd -24(%rbp), %xmm0
    addsd -16(%rbp), %xmm0
    movsd %xmm0, -32(%rbp)
    movsd -32(%rbp), %xmm0
    movsd %xmm0, -4(%rbp)
    movsd -4(%rbp), %xmm0
    cvttsd2si %xmm0, %eax
    movl %eax, -40(%rbp)
    movl -40(%rbp), %eax
    jmp main_return_exit
main_return_exit:
    movq %rbp, %rsp
    popq %rbp
    ret
.section .note.GNU-stack,"",@progbits
.section .rodata
.LC0:
    .double 0
.LC1:
    .double 3
```

Le jeux de tests

Tests unitaires

- Tests après chaque fonctionnalité implémentée
- Vérification progressive (arithmétique, if, boucles, etc.)
- Détection rapide des erreurs

```c
int main() {
    int a;
    a = 1 + 3;
    return a;
}
```

```c
void print_x(int x) {
    putchar(x);
}

int main() {
    print_x(42);
    return 0;
}
```

```c
int main()
{
    int a;
    a = 2;

    int v;
    v = 1;
    if (a == 1)
    {
        int b;
        b = 1;
    }
    else if (a == 2)
    {
        v = 5;
    }

    else
    {
        int i;
        i = 1;
    }

    return v;
}
```

Test Global (Démo)

- Programme combinant plusieurs fonctionnalités
- Validation globale (récursion, switch, I/O)
- Vérification du bon fonctionnement du code généré

```c
int fact(int n) {
    if (n <= 1) { return 1; }
    return n * fact(n - 1);
}

int square(int n) {
    return n * n;
}

void print_int(int n) {
    if (n >= 10) {
        print_int(n / 10);
    }
    putchar(n % 10 + '0');
}

int main() {
    int choice, charValue, intValue, result;

    choice = getchar();
    charValue = getchar();

    intValue = charValue - '0';

    switch (choice) {
        case 102:
            result = fact(intValue);
            break;
        case 115:
            result = square(intValue);
            break;
        default:
            result = 0;
    }

    print_int(result);
    putchar(10);

    return 0;
}
```

Démo

Merci !